

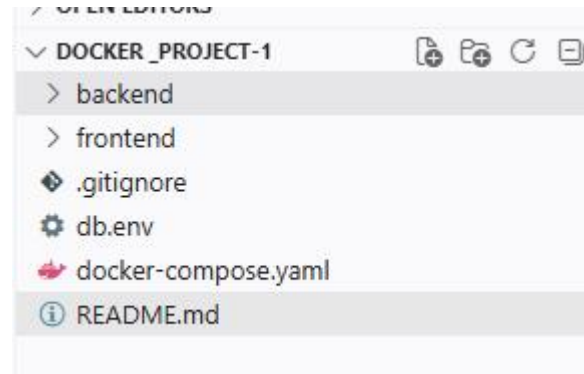
Project-1

Deployment of a 3-Tier Application on AWS EC2 using Docker and Docker Compose

Objective:

The objective of this project is to deploy a 3-tier application (React + FastAPI + PostgreSQL) on an AWS EC2 instance using Docker and Docker Compose. The application should be accessible through the EC2 Public IP address and all the services should communicate successfully.

Project Structure:



Project Architecture:

User --->frontend (React) --->Backend (Fast API) ---->Database(PostgreSQL)

EC2 Instance Creation:

Cloud Provider: AWS
Instance Type: t3.micro (Free Tier Eligible)
Operating System: Amazon Linux 2023
Region: Asia Pacific (Mumbai) - ap-south-1
Public IPv4 Address:15.252.123.236
Storage: 8 GB
Key Pair Name: docker_project-1_key_pair
Status: Running

✓	Name ↗	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...
✓	docker_project-1	i-08f8d41f14bf93424	Running	t3.micro	3/3 checks passed	ap-south-1b	ec2-15-252-123-236.ap...	15.252.123.236

Security Group Configuration:

Type	Port
HTTP	80
Custom TCP	8000
SSH	22

sg-00e9eb10f1cb3609c - launch-wizard-1

Details

Security group name
launch-wizard-1

Owner
354089337751

Security group ID
sg-00e9eb10f1cb3609c

Inbound rules count
3 Permission entries

Description
launch-wizard-1 created 2026-07-29T10:50:40.823Z

Outbound rules count
1 Permission entry

VPC ID
vpc-0f9eb15138390ecb2

Inbound rules | Outbound rules | Sharing | VPC associations | Related resources - new | Tags

Inbound rules (3)

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sgr-01ab9897f1d68847c7	IPv4	SSH	TCP	22	0.0.0.0/0	-
-	sgr-0617db82106c883bd	IPv4	Custom TCP	TCP	8000	0.0.0.0/0	-
-	sgr-023212683e9327481	IPv4	HTTP	TCP	80	0.0.0.0/0	-

Docker Installation and Docker Compose Installation:

Both was installed successfully on the EC2 instance.

```
[ec2-user@ip-172-31-8-200 ~]$ docker --version
Docker version 25.0.14, build 0bab007
```

```
[ec2-user@ip-172-31-8-200 ~]$ docker compose version
Docker Compose version v5.3.1
```

Cloning the Repository in EC2:

```
[ec2-user@ip-172-31-8-200 ~]$ git clone https://github.com/machhapavithra-2105/te-5th_and_6th_batch_assignments.git
Cloning into te-5th_and_6th_batch_assignments...
remote: Enumerating objects: 452, done.
remote: Counting objects: 100% (223/223), done.
remote: Compressing objects: 100% (156/156), done.
remote: Total 452 (delta 90), reused 162 (delta 62), pack-reused 229 (from 1)
Receiving objects: 100% (452/452), 26.06 MiB | 9.21 MiB/s, done.
Resolving deltas: 100% (114/114), done.
```

Docker Compose Configuration:

Docker Compose is used to manage all three containers.

```
[ec2-user@ip-172-31-8-200 Docker _Project-1]$ cat docker-compose.yml
services:
  db:
    image: postgres:16-alpine
    env_file:
      - ./db.env
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 3s
      retries: 20

  backend:
    build:
      context: ./backend
    env_file:
      - ./backend/.env
    depends_on:
      db:
        condition: service_healthy
    ports:
      - "8000:8000"

  frontend:
    build:
      context: ./frontend
    depends_on:
      - backend
    ports:
      - "80:80"

volumes:
  pgdata:
```

Running the Application:

```
[ec2-user@ip-172-31-8-200 Docker _Project-1]$ docker compose up -d --build
[+] Building 74.4s (29/29) FINISHED

=> [internal] load local bake definitions                                0.0s
=> => reading from stdin 1.20kB                                         0.0s
=> [frontend internal] load build definition from Dockerfile            0.1s

[+] up 7/7
✓ Image docker_project-1-frontend Built 74.4s
✓ Image docker_project-1-backend Built 74.4s
✓ Network docker_project-1_default Created 0.2s
✓ Volume docker_project-1_pgdata Created 0.0s
✓ Container docker_project-1-db-1 Healthy 6.6s
✓ Container docker_project-1-backend-1 Started 6.9s
✓ Container docker_project-1-frontend-1 Started 7.3s
```

Docker Containers:

```
[ec2-user@ip-172-31-8-200 Docker _Project-1]$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
d5278a21644f   docker_project-1-frontend-1        "/docker-entrypoint..." 2 minutes ago  Up 2 minutes  0.0.0.0:80->80/tcp, :::80
d78819b524d6   docker_project-1-backend-1         "uvicorn main:app --..." 2 minutes ago  Up 2 minutes  0.0.0.0:8000->8000/tcp, :
605cb3ba58dd   postgres:16-alpine                 "docker-entrypoint.s..." 2 minutes ago  Up 2 minutes (healthy)  0.0.0.0:5432->5432/tcp, :
```

Database Persistence:

PostgreSQL uses a named volume so that data is retained even after restarting the containers.

```
[ec2-user@ip-172-31-8-200 Docker _Project-1]$ docker volume ls
DRIVER      VOLUME NAME
local       docker_project-1_pgdata
```

Not secure 15.252.123.236

YouTube Maps Console Home | Co... AWS Sessions SRE 5... (4) WhatsApp AWS VPC Practical... The 6 Pillars of the... Devops

Inventory Tracker

Total: 4

Search by id, name or description...

Add Product

ID	Name	Description	Price
Quantity			

Add

Track. Manage. Grow.

Streamline your inventory with smart product management that scales with your business.

POWERED BY SIVA

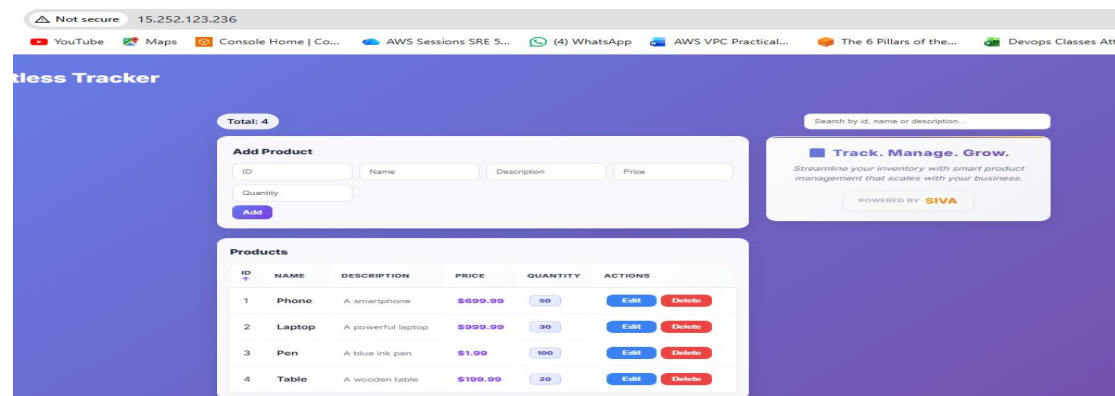
Products

ID	NAME	DESCRIPTION	PRICE	QUANTITY	ACTIONS
1	Phone	A smartphone	\$699.99	50	Edit Delete
2	Laptop	A powerful laptop	\$999.99	30	Edit Delete
3	Pen	A blue ink pen	\$1.99	100	Edit Delete
4	Table	A wooden table	\$199.99	20	Edit Delete

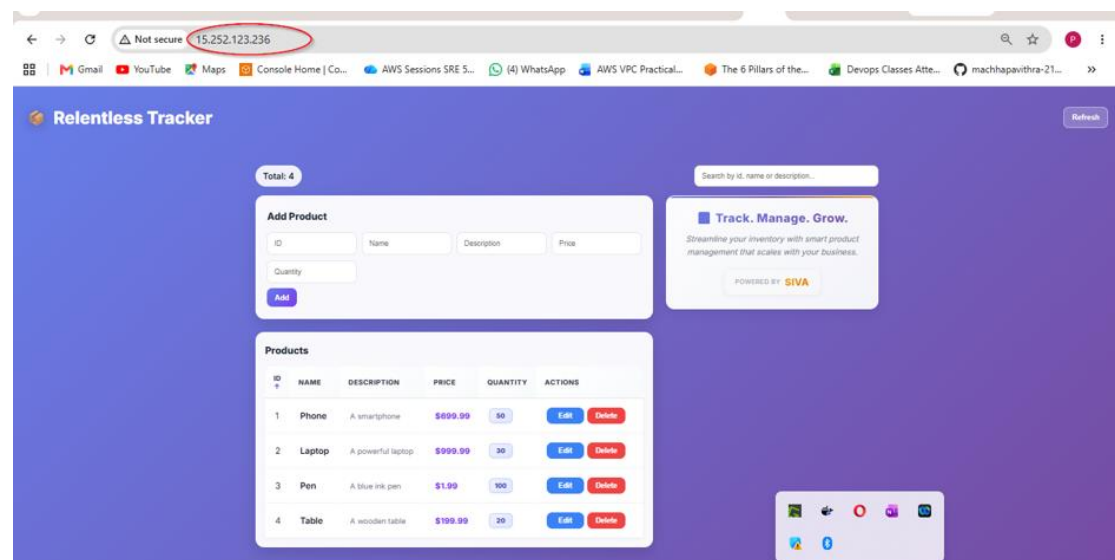
To verify database persistence, the application was stopped using `docker compose down` and started again using `docker compose up -d`. After restarting the containers, the previously stored product data was still available in the application. This confirms that PostgreSQL data is stored in a named Docker volume and survives container restarts.

```
[ec2-user@ip-172-31-8-200 Docker _Project-1]$ docker compose down
[+] down 4/4
✓ Container docker_project-1-frontend-1 Removed
✓ Container docker_project-1-backend-1 Removed
✓ Container docker_project-1-db-1 Removed
✓ Network docker_project-1_default Removed
[ec2-user@ip-172-31-8-200 Docker _Project-1]$ docker volume ls
DRIVER      VOLUME NAME
local       docker_project-1_pgdata
[ec2-user@ip-172-31-8-200 Docker _Project-1]$ docker compose up -d
[+] up 4/4
✓ Network docker_project-1_default Created
✓ Container docker_project-1-db-1 Healthy
✓ Container docker_project-1-backend-1 Started
✓ Container docker_project-1-frontend-1 Started
[ec2-user@ip-172-31-8-200 Docker _Project-1]$ |
```

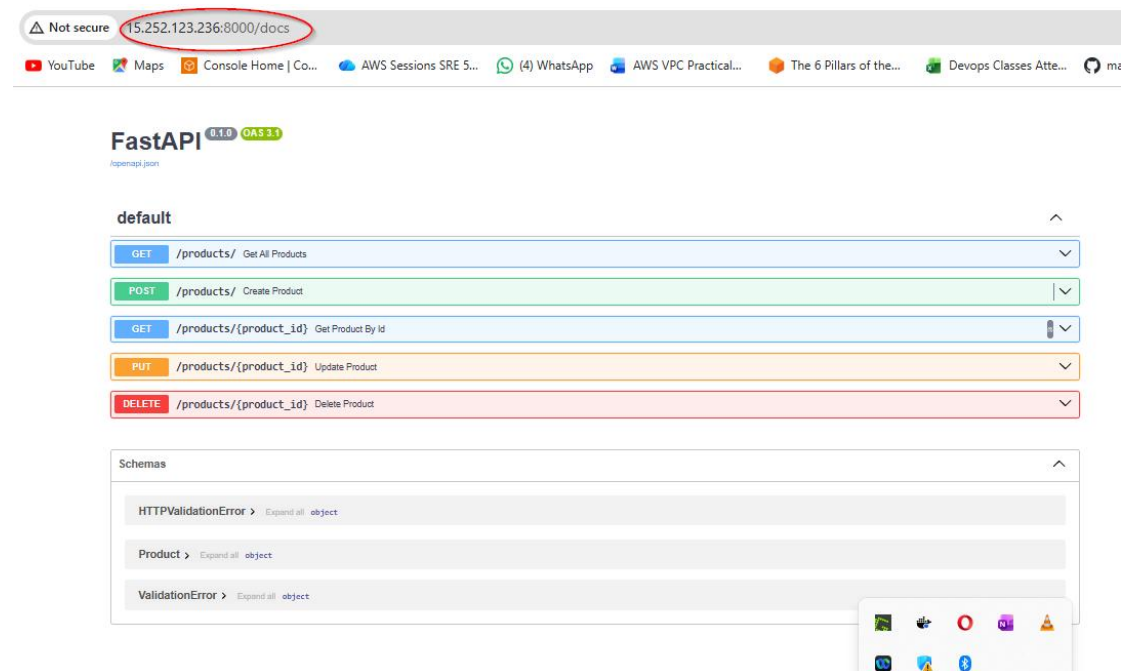
```
[ec2-user@ip-172-31-8-200 Docker _Project-1]$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
f26cecdc084    docker_project-1-frontend-1        "/docker-entrypoint..." 2 minutes ago  Up 2 minutes  0.0.0.0:80->80/tcp, :::80->80/tcp
6c6e22d69aca    docker_project-1-frontend-1        "/docker-entrypoint..." 2 minutes ago  Up 2 minutes  0.0.0.0:80->80/tcp, :::80->80/tcp
6c6e22d69aca    docker_project-1-backend-1        "uvicorn main:app --..." 2 minutes ago  Up 2 minutes  0.0.0.0:8000->8000/tcp, :::8000->8000/tcp
a4750d75d218    postgres:16-alpine                "docker-entrypoint.s..." 2 minutes ago  Up 2 minutes (healthy)  0.0.0.0:5432->5432/tcp, :::5432->5432/tcp
a4750d75d218    docker_project-1-db-1              "docker-entrypoint.s..." 2 minutes ago  Up 2 minutes (healthy)  0.0.0.0:5432->5432/tcp, :::5432->5432/tcp
```



Frontend Deployment:



Backend Verification:



Conclusion:

Successfully deployed a 3-tier application on AWS EC2 using Docker and Docker Compose. The application is accessible through the EC2 Public IP address and all the project requirements have been satisfied.